Not content with using the platform-specific C APIs for handling multithreading, C++ programmers have looked to their class libraries to provide object-oriented multithreading facilities. Application frameworks such as MFC and general-purpose C++ libraries such as Boost and ACE have accumulated sets of C++ classes that wrap the underlying platform-specific APIs and provide higher-level facilities for multithreading that simplify tasks. Although the precise details of the class libraries have varied considerably, particularly in the area of launching new threads, the overall shape of the classes has had a lot in common. One particularly important design that's common to many C++ class libraries, and that provides considerable benefit to the programmer, has been the use of the *Resource Acquisition Is Initialization* (RAII) idiom with locks to ensure that mutexes are unlocked when the relevant scope is exited.

For many cases, the multithreading support of existing C++ compilers combined with the availability of platform-specific APIs and platform-independent class libraries such as Boost and ACE provide a solid foundation on which to write multithreaded C++ code, and as a result there are probably millions of lines of C++ code written as part of multithreaded applications. But the lack of standard support means that there are occasions where the lack of a thread-aware memory model causes problems, particularly for those who try to gain higher performance by using knowledge of the processor hardware or for those writing cross-platform code where the actual behavior of the compilers varies between platforms.

1.3.2. Concurrency support in the new standard

All this changes with the release of the new C++11 Standard. Not only is there a brand-new thread-aware memory model, but the C++ Standard Library has been extended to include classes for managing threads (see chapter 2), protecting shared data (see chapter 3), synchronizing operations between threads (see chapter 4), and low-level atomic operations (see chapter 5).

The new C++ Thread Library is heavily based on the prior experience accumulated through the use of the C++ class libraries mentioned